Конфиденциально

Отчет о тестировании на проникновение

# Внешний периметр организации “Яндекс Практикум (дипломный стенд)”

# 1. Введение

## 1.1. Информация о проекте

Наименование организации: Яндекс Практикум

Дата: 2025-06-07 - 2024-10-07

Перечень ресурсов: diploma-web-861easec.seccheck.ru

**Классификация уязвимостей по уровню критичности**

Все уязвимости, описанные в отчете, были классифицированы в соответствии с определенными категориями опасности.

Определение уровней критичности уязвимостей:

ИНФОРМАЦИЯ – данные уязвимости не несут прямой угрозы, однако они отмечены, поскольку отражают процесс тестирования и в будущем могут влиять на уровни безопасности внедряемых решений.

НИЗКИЙ – уязвимости данного типа не несут прямой угрозы конфиденциальности, целостности или доступности данных, обрабатываемых системой. Ошибки такого рода обычно позволяют или облегчают выявление слабых мест платформы, например, путем раскрытия дополнительной информации о работающем сервисе.

СРЕДНИЙ – уязвимости данного типа приводят к потере конфиденциальности, целостности или доступности обрабатываемых данных, но только при обстоятельствах, существенно ограничивающих спектр возможных атак.

ВЫСОКИЙ – эти уязвимости представляют собой неминуемую и значительную угрозу конфиденциальности, целостности или доступности обработанных данных, а также дополнительные обстоятельства, которые должны произойти для успеха атаки.

КРИТИЧЕСКИЙ – эти уязвимости представляют собой прямую и весьма вероятную угрозу массовой потери конфиденциальности, целостности или доступности обрабатываемых данных.

**Общая система оценки уязвимостей**

Общая система оценки уязвимостей (CVSS) — это отраслевой стандарт для оценки серьезности уязвимостей на основе конкретных, определенных показателей, таких как требуемые привилегии, вектор атаки, сложность атаки, влияние успешной атаки. Универсальность методологии помогает сравнивать уязвимости между собой, упрощая тем самым управление большим количеством уязвимостей.

Меры представляют собой оценку, которую должен анализировать владелец испытуемого, обладающий полными знаниями в области бизнеса, что позволяет ему скорректировать классификацию серьезности уязвимости.

В силу своего характера с помощью тестов на проникновение невозможно выявить все существующие уязвимости в рассматриваемых системах и приложениях. Тем не менее, они позволяют быстро и эффективно выявлять

основные уязвимости, которые могут быть обнаружены и использованы третьими лицами.

Процесс управления безопасностью состоит из внедрения надлежащих процедур и рабочих процедур, обновляемых по мере разработки новых платформ и с учетом обслуживания существующих систем, чему способствуют периодические формальные и технические проверки внедренных решений.

## Модель нарушителя

Потенциальным нарушителем считается лицо или группа, которые могут осуществлять угрозы безопасности и пытаться скомпрометировать информационные ресурсы Системы и нанести ущерб интересам Компании, действуя в сговоре или отдельно, со злым умыслом или без него.

1. Результаты тестирования

1.1. Общая статистика

| Критичность | Amount |
|---|---|
| Высокая | 4 |
| Средняя | |
| Низкая | |
| Всего обнаружено | 4 |

1.2. Резюме

| Название уязвимости | Критичность | Вероятность | Итоговая критичность |
|---|---|---|---|
| LFI | Высокая | Высокая | Высокая |
| Небезопасная десериализация | Высокая | Высокая | Высокая |

| | | | |
|---|---|---|---|
| SQLi | Высокая | Высокая | Высокая |
| SSTI | Высокая | Высокая | Высокая |

Список уязвимостей:

1. LFI (Local File Inclusion).
2. SQL-инъекция
3. Небезопасная десериализация
4. SSTI

## Уязвимость: Local File Inclusion

Критичность: Высокая

Вероятность: Высокая

Итоговый риск: Высокий

**Числовой показатель критичности и вектор CVSS: 10**
(AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:L/A:H)

**Описание**

**LFI** - это возможность использования и выполнения локальных файлов на серверной стороне. Уязвимость позволяет удаленному пользователю получить доступ с помощью специально сформированного запроса к произвольным файлам на сервере, в том числе содержащую конфиденциальную информацию.

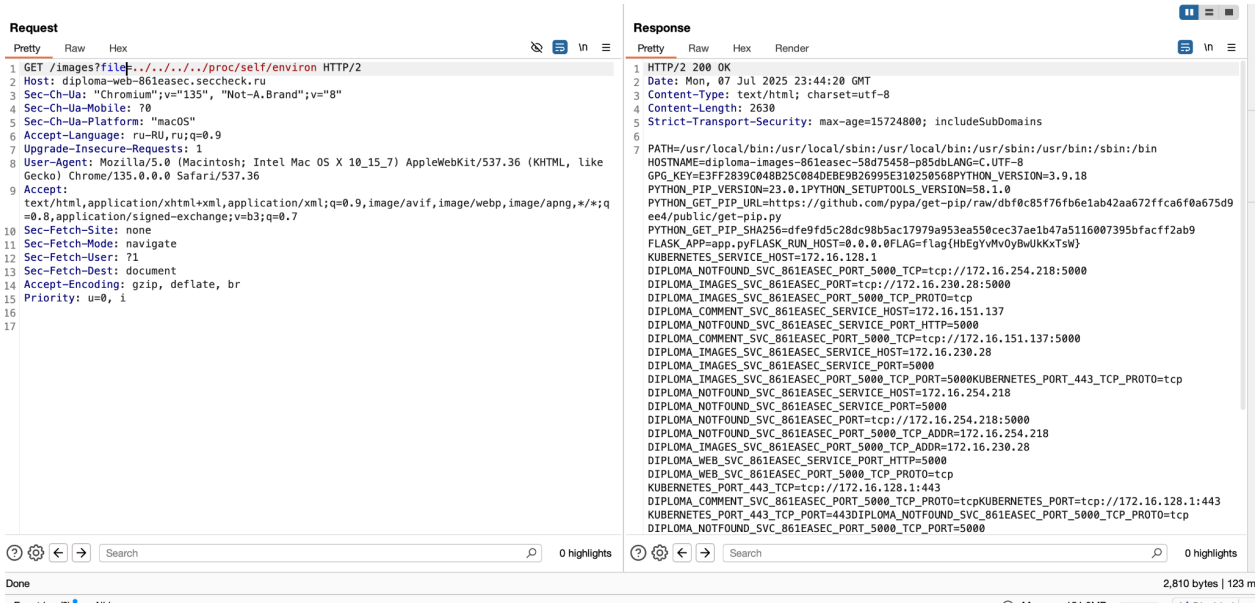
**Влияние**

Кража конфиденциальной информации, возможно развитие до RCE и загрузки обратного шелл кода, DoS

**Шаги для воспроизведения**

В данной ситуации необходимо найти GET параметр /images?file=post0.jpg. Ищем ее либо с помощью утилит ffuf/dirsearch, либо через перехват запросов в Burp Suite.
Далее пробуем разные полезные нагрузки, чтобы найти LFI (/etc/passwd, /etc/shadow и т.д)
Для того, чтобы найти необходимую чувствительную информацию/флаг

потребовалась нагрузки *“/images?file=../../../../proc/self/environ”.* В результате чего в ответе сервера мы получаем различные данные в которых содержится необходимая нам информация.




**Рекомендации по устранению**

- **Нормализация путей.** Убедись, что пользовательский ввод нормализован, то есть удалены последовательности ../, ./ и другие символы, которые могут быть использованы для выхода за пределы разрешённых директорий.
- **Валидация пользовательского ввода.** Запрети использование ../ в пользовательском вводе, а также других символов, которые могут быть интерпретированы как переход по каталогам.
- **Ограничение доступа.** Настрой жёсткие ограничения для директорий, к которым сервер имеет доступ, а также используй виртуальные sandbox для изоляции загружаемых файлов.
- **Минимизация прав.** Запускай серверное приложение от пользователя с минимальными правами.

# Уязвимость: SQL-инъекция

Критичность: Высокая

Вероятность:  Высокая

Итоговый риск: Высокий

**Числовой показатель критичности и вектор CVSS: 10**

(AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:L/A:H)

**Описание**

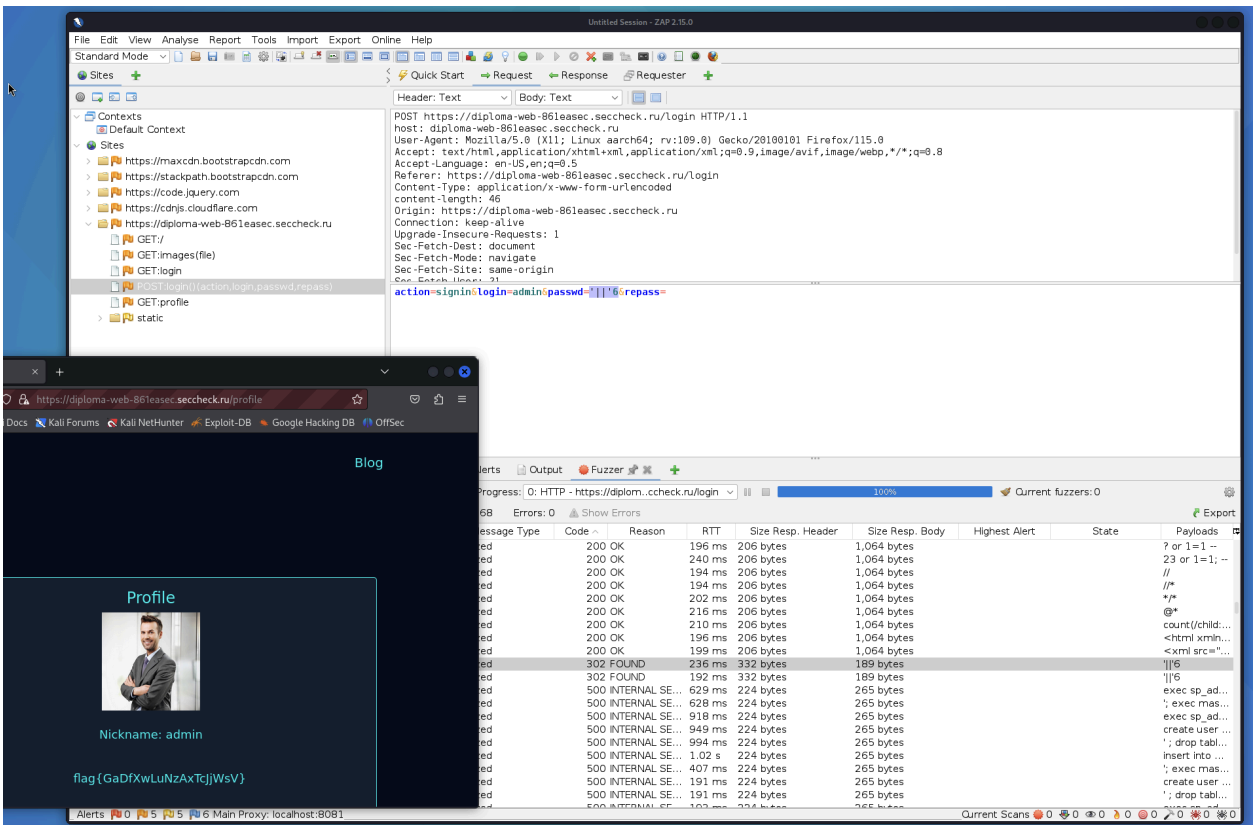
**SQL Injection** — уязвимость, при которой злоумышленник вставляет вредоносный SQL-код в запросы к базе данных. Код выполняется на сервере — это позволяет манипулировать данными, обходить авторизацию, извлекать или изменять конфиденциальную информацию.

**Влияние**

- Доступ к базе данных, паролям, перепискам пользователей, платёжной информации.
- Изменение данных в базе — злоумышленник может создать пользователя с правами администратора. Это грозит полной компрометацией веб-приложения.
- Удалённое выполнение кода на сервере.
- Получение доступа к профилю через обход аутентификации.

**Шаги для воспроизведения**

Для этого потребуется утилита ZAP, необходимо запустить сканер для поиска уязвимости на странице аутентификации с различными инъекциями (предпочтительнее SQL), так, чтобы логин был admin.
Ждем какое-то время и анализируем ответы от сервера, находим ответ от сервера 302, вводим в поле ввода пароля ‘||’6 и получаем доступ к профилю администратора и еще одному флагу.




## Рекомендации по устранению

- Параметризированные запросы — средство предварительной компиляции инструкций SQL. Здесь значения введённых данных передаются как параметры, а не внедряются в текст запроса. Этот метод позволяет базе данных распознавать код и отличать его от введённых данных.
- ORM, или Object-Relational Mapping, тоже исключает возможность внедрения пользовательского ввода в SQL-запрос. Этот подход позволяет работать с базой данных на уровне объектов. ORM-фреймворки предоставляют средства для определения структуры данных в виде классов и их связей. Ещё они автоматически генерируют SQL-запросы для сохранения и извлечения данных из базы. ORM упрощает и ускоряет разработку приложений, делает код понятным и лёгким для поддержки.
- Белый список разрешённых значений.

# Уязвимость: Небезопасная десериализация

Критичность: Высокая

Вероятность: Высокий

Итоговый риск: Высокий

**Числовой показатель критичности и вектор CVSS: 9.9**

(AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H)

**Описание**

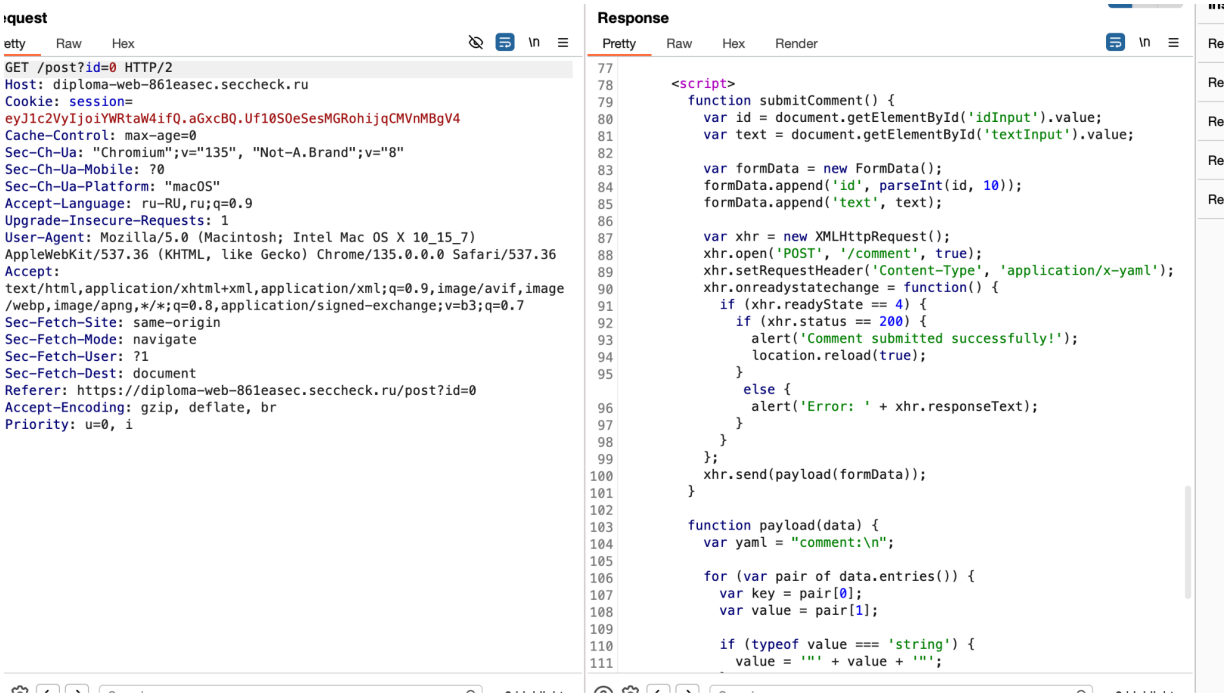
Небезопасная десериализация — это ситуация, когда сайт десериализует данные, которые контролирует пользователь.

**Влияние**

- RCE,
- повышение привилегий в системе,
- получение неавторизованный доступ к файлам и функциям веб-приложения.

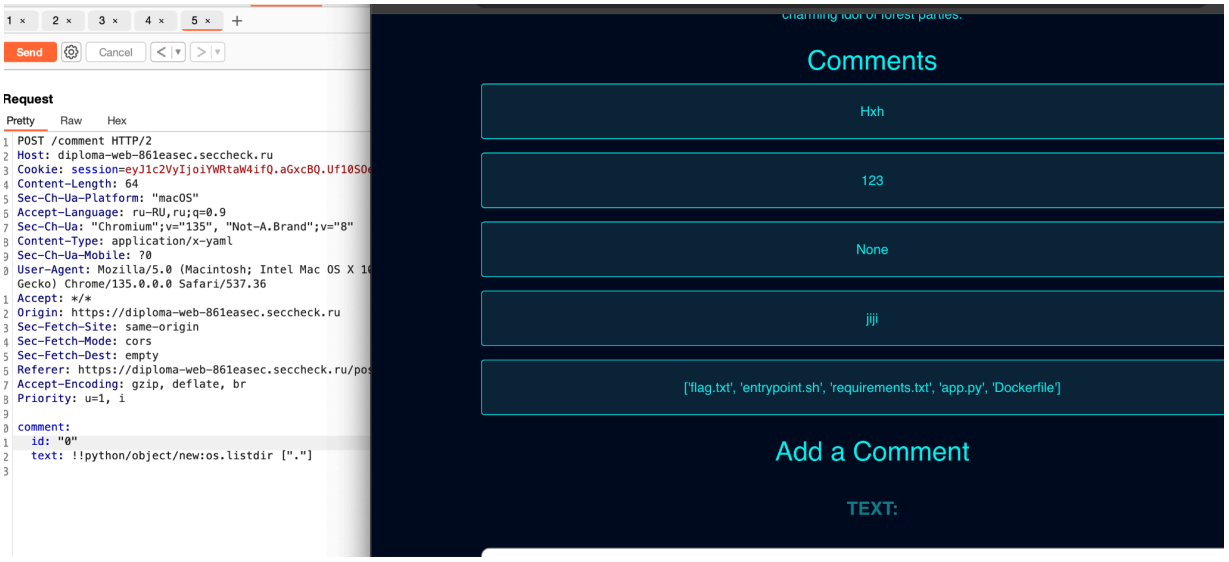
**Шаги для воспроизведения**

Первое, что необходимо сделать это понять, что комментарии сериализуются с помощью Yaml. Это можно сделать посмотрев в html коде странице найдя скрипт обработки, либо при перехвате запроса в Burp Suite. Далее перехватываем запрос при отправке комментария. Подбираем необходимую инъекцию и вставляем в тело запроса.




1.

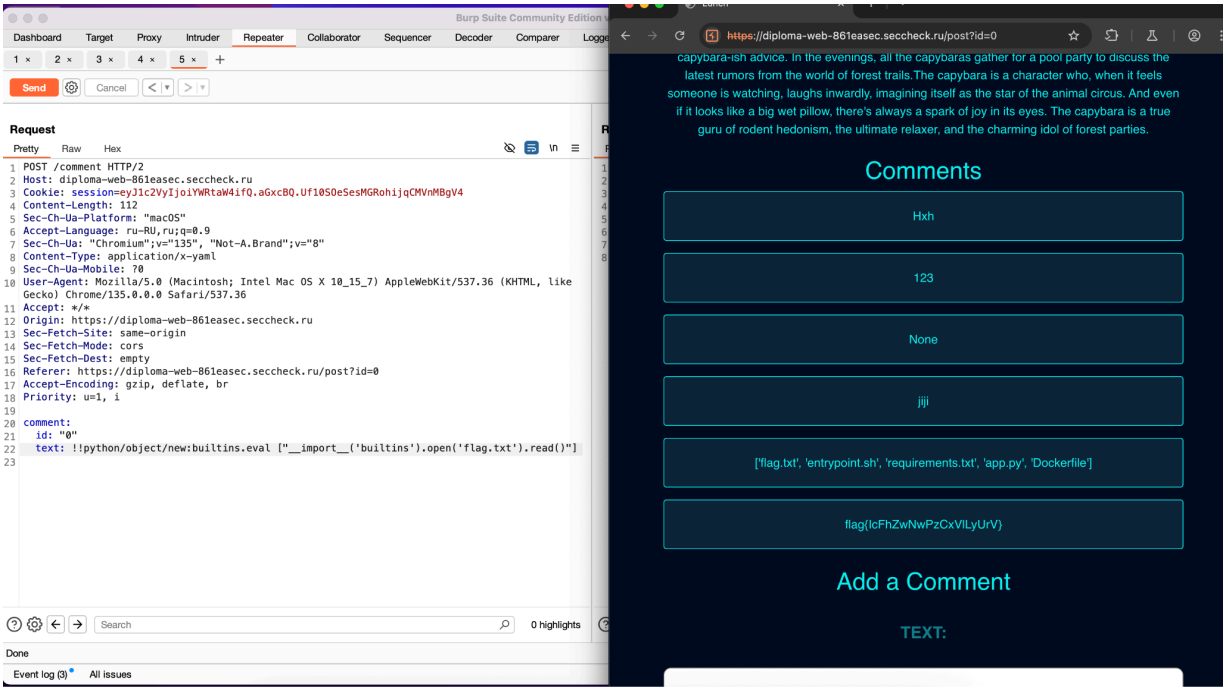
comment:
id: "0"
text: !!python/object/new:os.listdir [".."]




Нашли директории и файл flag.txt

2.

comment:
id: "0"
text: !!python/object/new:builtins.eval ["__import__('builtins').open('flag.txt').read()"]




Получили содержимое файла flag.txt.

### Рекомендации по устранению

- Не десериализуйте пользовательский ввод без серьёзной необходимости.
- Опасность возможных эксплоитов и сложность в построении защиты перевешивают преимущества.
- Если десериализуете данные из ненадёжных источников, несколько раз проверяйте подлинность данных.
- Например, используйте цифровую подпись, чтобы убедиться в их целостности. Такие проверки необходимо выполнять до начала десериализации, иначе они не будут эффективными.
- Не используйте стандартные функции десериализации.
- Создавайте собственные методы. Пусть у каждого сериализуемого класса будет отдельный метод. Так будет проще контролировать открытые атрибуты. По возможности в PHP можно использовать функции json_encode и json_decode вместо serialize и unserialize, а в Python — библиотеку json вместо pickle.
- Ограничивайте типы при десериализации.
- Если возможно, определяйте белый список классов, разрешённых для десериализации. Это позволяет работать только с безопасными и ожидаемыми типами объектов. В Java можно использовать ObjectInputFilter, чтобы ограничить типы данных при десериализации. Так

разработчик определит строгие фильтры на объекты, которые могут быть десериализованы
- Валидируйте данные.
- Перед десериализацией тщательно валидируйте данные. Так вы убедитесь, что они находятся в ожидаемом формате и содержат безопасные значения.
- Помните, что причина возникновения уязвимости — это десериализация пользовательского ввода.

## Уязвимость: SSTI

Критичность: Высокая

Вероятность: Высокая

Итоговый риск: Высокий

**Числовой показатель критичности и вектор CVSS: 10**
(AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H)

**Описание**

**SSTI** — возникает, когда пользовательский ввод некорректно обрабатывается веб-приложением при работе с шаблонами. Это позволяет злоумышленнику внедрить и исполнить произвольный код на сервере.

**Влияние**

SSTI позволяет злоумышленнику не только выполнять команды на сервере, но и получить полный доступ к серверной инфраструктуре. Это может включать:
- Доступ к конфиденциальной информации и данным пользователей.
- Возможность установки бэкдоров или вредоносного ПО на сервере.
- Потенциальное использование сервера для атак на другие системы.

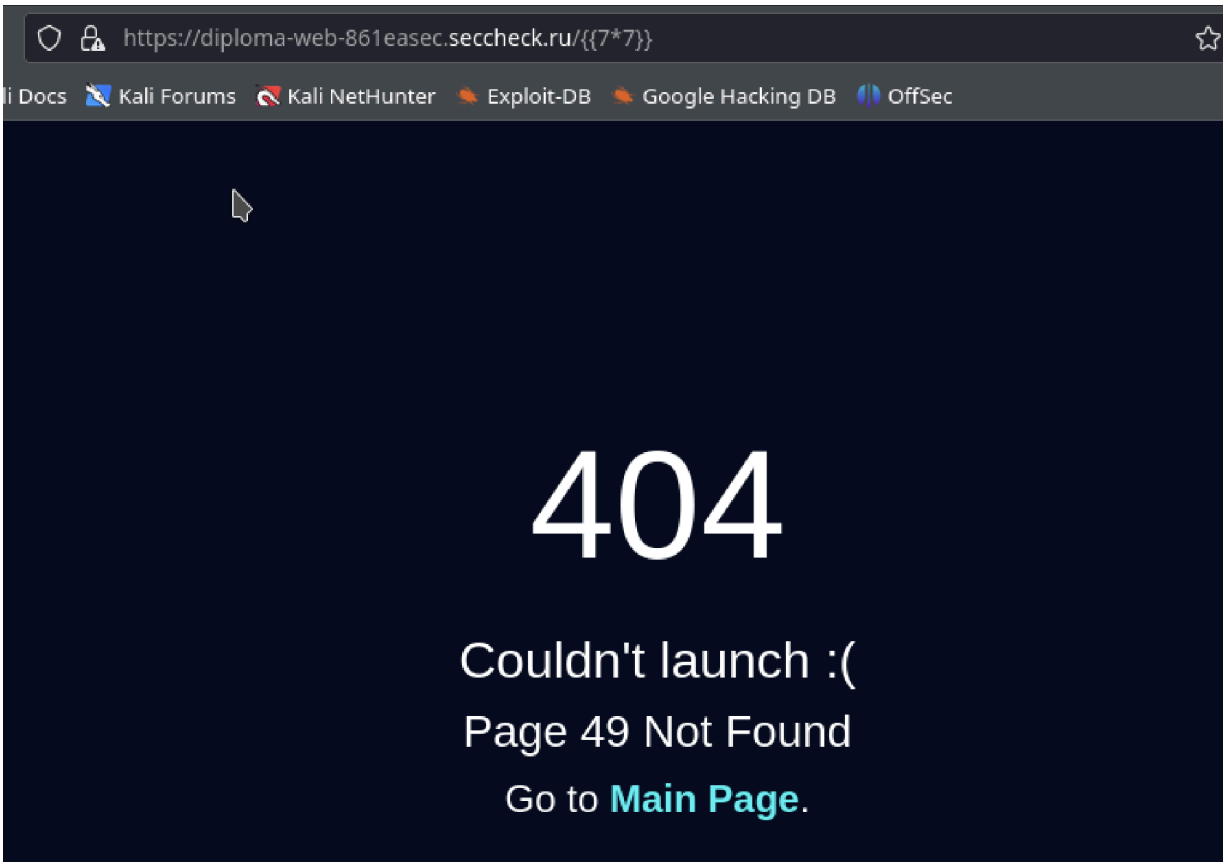
Импакт такой атаки может быть колоссальным, включая потерю данных, повреждение репутации компании и вытекающие юридические последствия.

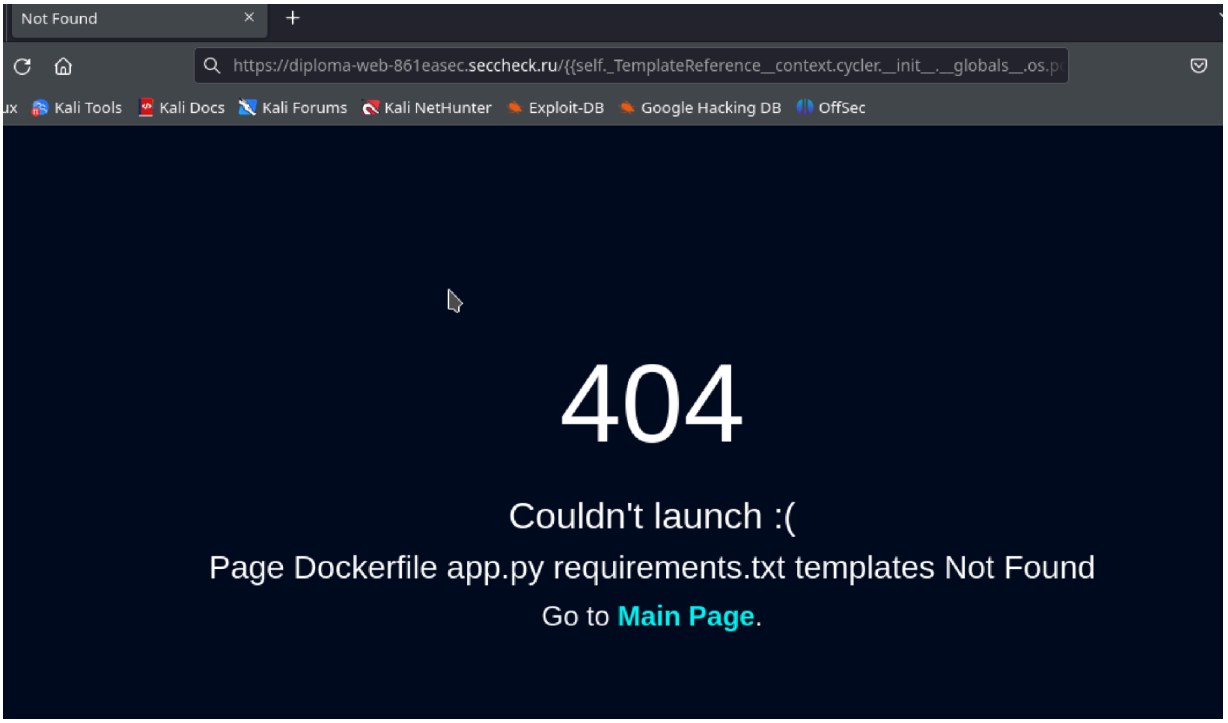
**Шаги для воспроизведения**

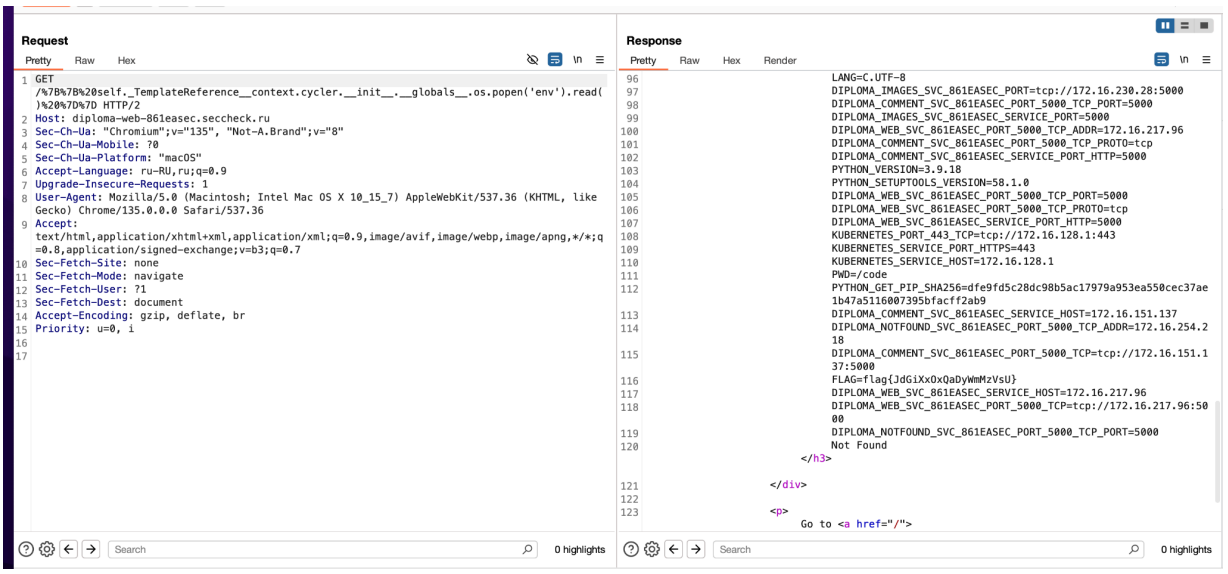
Находим что сайт чувствителен к SSTI методом перебора (Burp Suite/ffuf/dirsearch/ZAP). В данном случае при вводе {{7*7}} получаем ответ

“49 страница не найдена”. Понимаем, что используется Jinja2. Далее необходимо отправить запрос: GET /{{self._TemplateReference__context.cycler.__init__.__globals__.os.popen('env').read()}}

Таким образом мы получаем ответ с переменными окружения, в которых находится еще один флаг.






## Рекомендации по устранению

- Валидировать пользовательский ввод.
- Использовать logic-less шаблонизаторы.
- Использовать sandbox-режим, чтобы изолировать выполнение шаблонов.
- Отключать небезопасные методы в конфигурации движков.